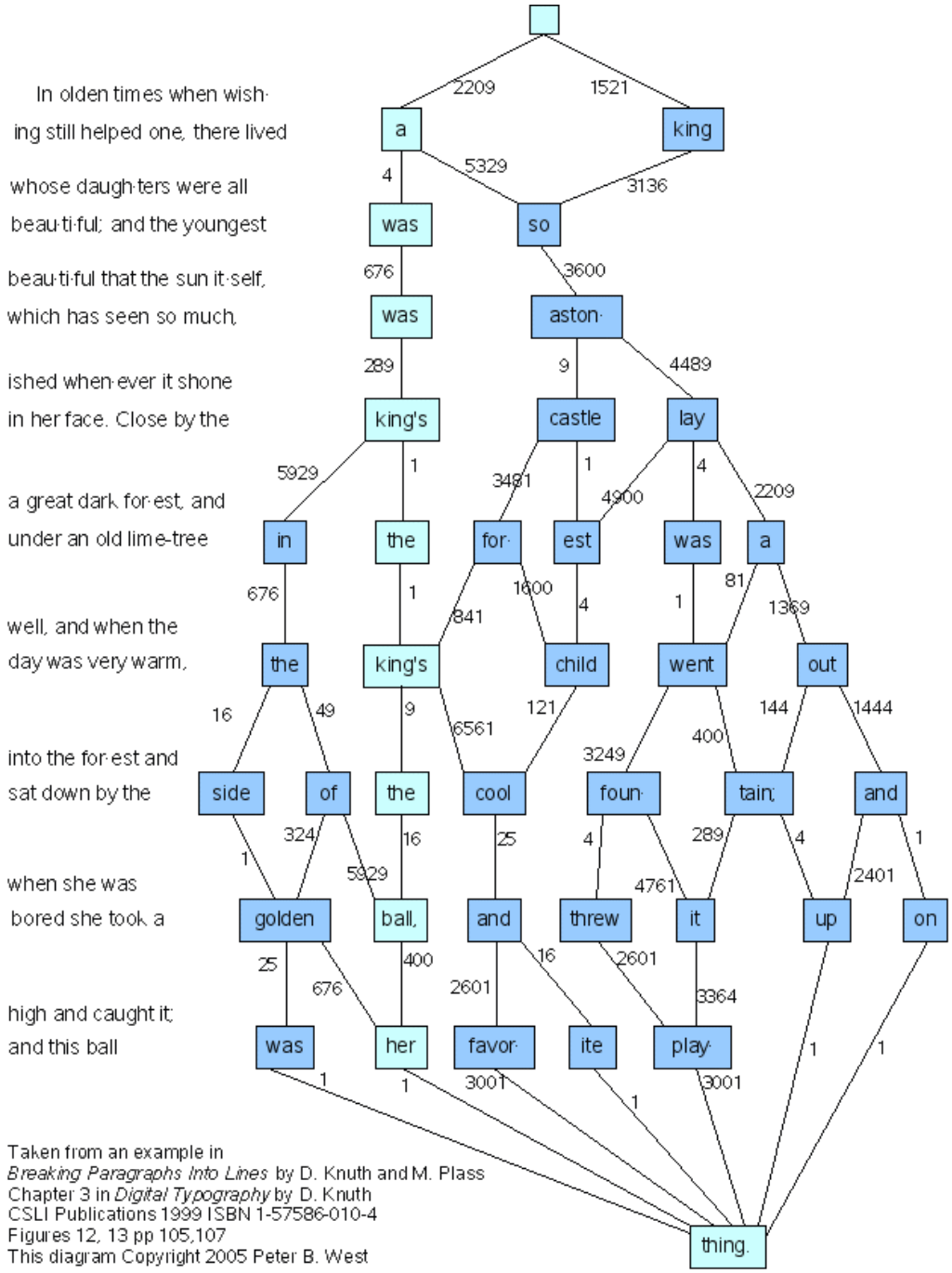
Home page

Project Summary page

Knuth & Plass line-breaking Revisited

Paragraphs mean Knuth and Plass, and the line-breaking algorithm implemented in TEX. The FOP guys have implemented Knuth/Plass paragraph breaking, thanks to Luca Furini, some time ago. Knuth & Plass are pretty much the only game when it comes to publicly-available paragraph line-breaking, ever since they published the article *Breaking Paragraphs Into Lines* in *Software - Practice and Experience 11 (1981) 1119-1184*. This article was republished in *Digital Typography (Donald E. Knuth. CSLI Lecture Notes No. 78 ISBN 1-57586-010-4 1999)* as Chapter 3.

My initial requirement is to translate the algorithm from the Pascal milieu of Knuth to Java. In particular, as I am intending to make the maximum use of the Java 2D facilities of the language, into an `AttributedString`, `BreakIterator`, `GlyphVector` environment. This is a sketch of my current understanding of the algorithm. Speaking of sketches, here is a diagram conflated from a couple of the diagrams in the Knuth/Plass article. It represents a network of feasible break-points generated by the algorithm for one of the paragraphs in the longer text example used in the article for illustration. The final set of line-breaks is represented by the pale green boxes; all other feasible break-points that were discovered are represented by the blue boxes.



The whole of the paragraph is being set in lines of constant width, with a *tolerance* of 1 (of which more later.) The text being considered commences with the unadorned text on the left. Down the left hand side is all of the text in which no feasible line breaks were found. At the right, at the end of each of the runs of unbroken text, are those elements which are feasibe break-points. The text is shown with all of the possible hypenation points marked with a middle dot, as is commonly used in dictionaries. Only in "lime-tree" is there a manifest hyphen. As the algorithm starts, a so-called *active node*, representing the beginning of the text, has been introduced to the otherwise empty list of *active nodes*. The algorithm now works through the paragraph, considering each possible break-point. Possible break-points, in English text, are the spaces between the words, explicit hyphens and possible hyphenation points, represented in some implementation-dependent way.

What is indicated in the diagram is that no feasible break-points were found in the text starting "*In olden times when wish·ing still helped one, there lived*". How is this determined? For each possible break-point, for each active node, the width of the text from the node to the possible break-point is calculated. If this length "fits" in the available line width, this is a feasible break-point.

The precise meaning of "fits" will be discussed later, but for now simply assume that there is some *elasticity* in the definition of "fits". One of the limits of this elasticity is defined by the *tolerance*. No break-points are available in the intial run of text because the amount of "stretching" required to fit

the text on a line by itself would exceed the tolerance. At the words "a" and "king" in "a king" however, feasible break-points are found.

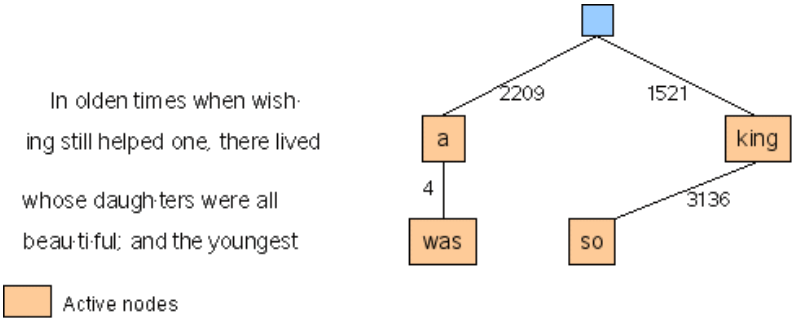
Some measure of the *goodness*, or correspondingly, the *badness*, of a given break-point is required. This measure is related to the degree of stretching of shrinking of the inter-word space that is required to get the work to fit on the line, and a few other parameters that will be discussed later. The results of these calculations is shown on the figure as *demerits*, or measures of badness of fit. They show that, by these calculations, "king" is a better place to break than "a". Both of these feasible break-points are added as nodes to the active nodes list. As a node is added, a pointer to the best active node (i.e. break-point) leading to this node is added. For both "a" and "king", that optimal break-point is the beginning of the paragraph.

The optimality horizon

When the next possible break-point (the space after "whose") is considered, it is compared with each of the active nodes. The first node is the beginning of the paragraph. The text from the beginning of the paragraph up to and including "whose" is too wide to be squeezed into one line, in spite of the fact that the elasticity of the inter-word spaces allows for a certain amount of shrinking, as well as stretching. Therefore, the beginning of the paragraph is removed from the list of active nodes. The text "king whose" is too short to fit on a line following a break at the node "a", just as "whose" is too short to fit on a line following a break at "king". The same holds for all of the text "whose daughters were all beautiful; and the youngest".

The next feasible break-point turns out to be following "was" with respect to the active node "a" It is not, however, feasible to break at "was" if the previous line ended with "king". A node is added for "was" with a link pointing back to "a". The demerits for this line are calculated (4) and added to the total demerits for the preceding node (here 2209) to give the total demerits of a break at this point.

When the next possible break-point, after "so", is considered, it is found that a break is possible relative to both "a" and "king". The demerits are calculated for breaks relative to each node. For "so"-"a" the total demerits are 5329+2209 = 7538; for "so"-"king they are 3136+1521 = 4657. (See the network diagram above.) Now the optimal path to this feasible break-point is determined, and an active node it added for the "so"-"king" path with the local and total demerits. Then, crucially for the algorithm, the "so"-"a" path is forgotten. This situation is represented in the following diagram.

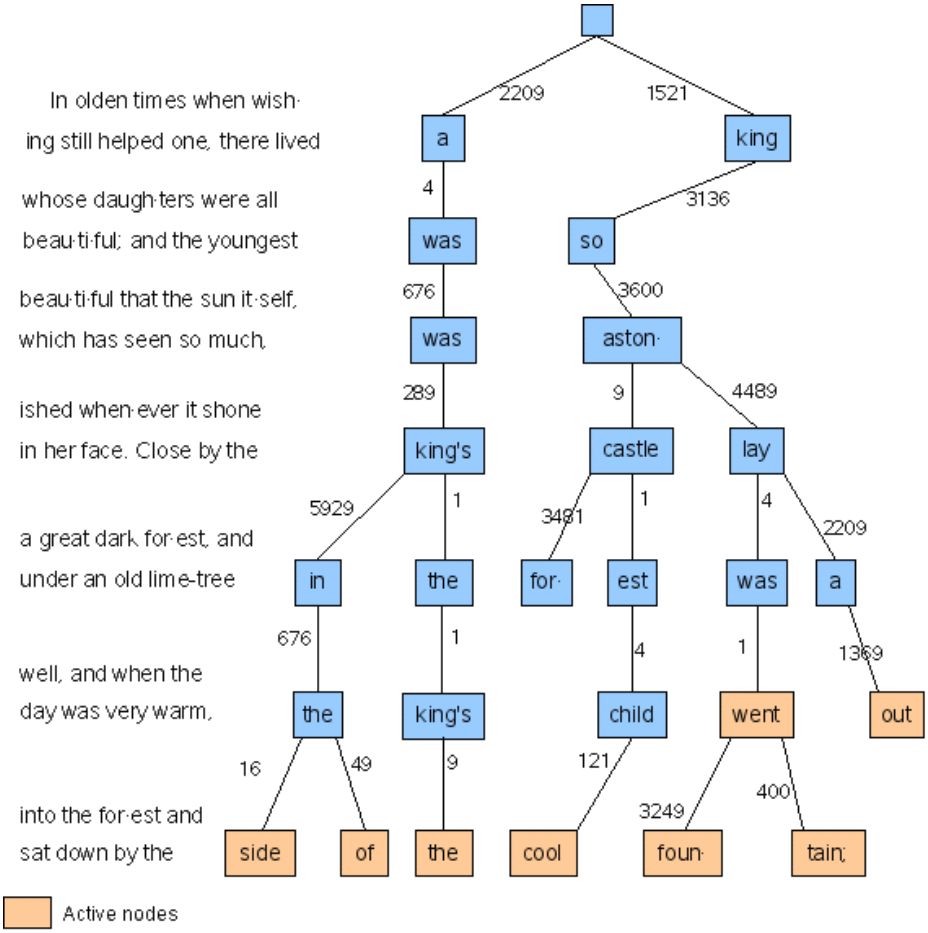


The optimality principle and the one-line horizon

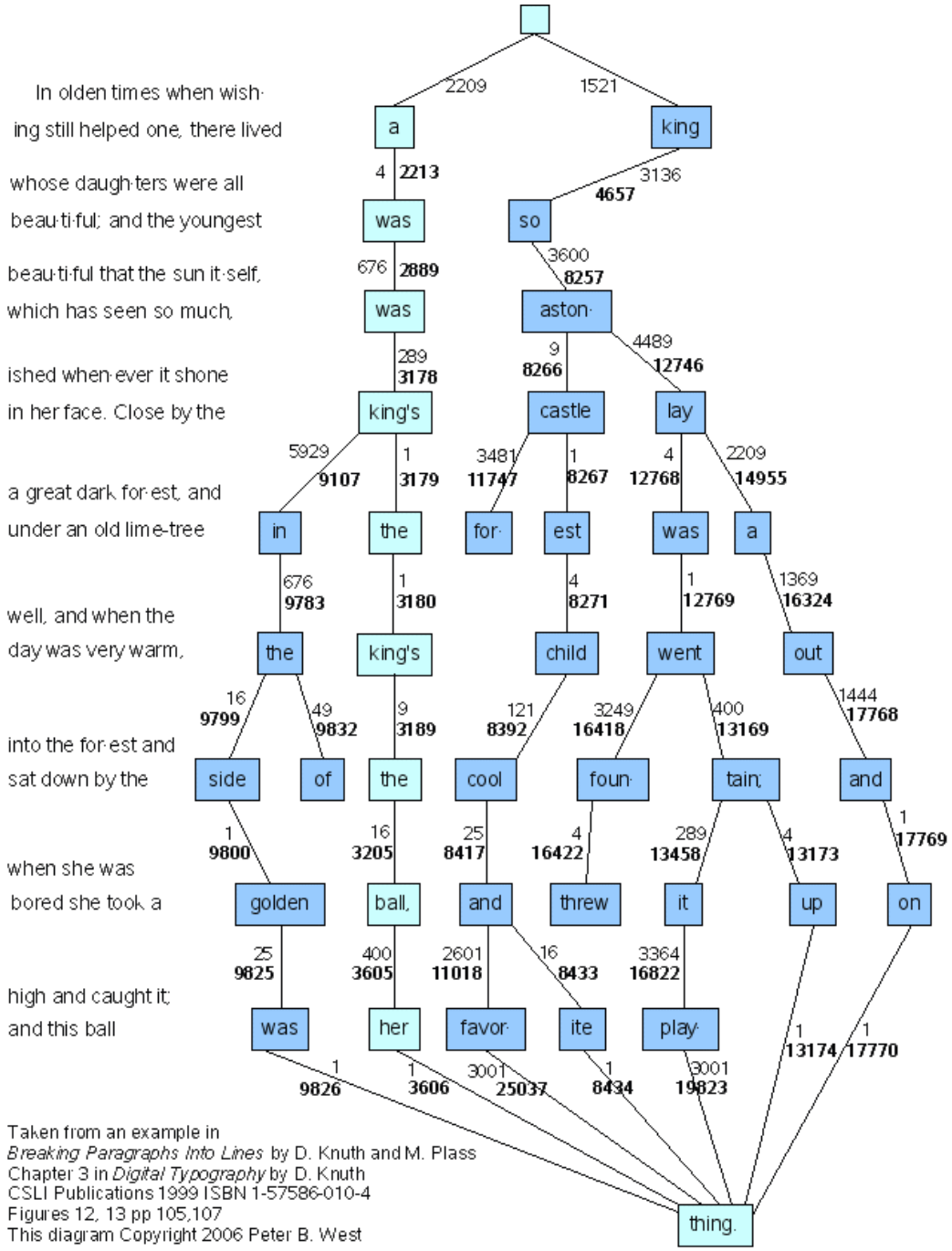
This algorithm is an application of mathematical dynamic programming. (Don't ask me.) One of the principles, it seems, is that, in the optimal solution, all partial solutions are also optimal. So that, in an optimally set paragraph, each sub-paragraph *from the beginning of the text* is also optimal. It is this principle that allows us to discard sub-optimal solutions at each feasible break-point, as we construct the graph of nodes. It also allows us to discard the nodes which are no longer within the horizon of our consideration. We effectively need only consider one line at a time, and for each feasible break-point node, only one prior break-point is retained, which will lead back to the beginning of the paragraph. To be more precise, the algorithm works with a "sliding window" one line wide, which is being dragged along by the right-hand side, advancing as each new possible break-point is considered, and skipping over the runs of text in which no break-point is feasible.

In the article, K&P describe the situation when the algorithm considers the break-point after "fount·ain;", i.e. the break-point after "tain;" in the diagram. There are eight active nodes at this stage; "child", "went", "out", "side", "of", "the", "cool" and "fount·". Notice that the text "into the

forest and sat down by" contains no feasible break-points, so it must be placed unbroken on a line. The node "king's" is no longer active, because the distance between it an the previously considered break-point was too great to fit on a line. The break-point after "tain;" is now assessed against each of the active nodes. The distance to "child" is too great, so "child" is removed from the active nodes list. Feasible break-points are found with respect to "went" and "out", and, predictably, all other possible lines are too short. The demerits of the line breaking at "went" are 400; that breaking at "out", 144. (See the full network diagram above.) However, the optimal paths to both of these nodes pass through "lay". The respective demerits to that common point are 1 + 4 for "went", and 1369 + 2209 for "out". (The algorithm simply keeps the total demerits for the optimal path to each node in the node itself, so it does not need to perform this illustrative analysis.) The total demerits for the path through "went" are therefore lower. Once again, the algorithm throws away the information about the rejected paths. After considering the break at "tain;", the graph looks as follows.

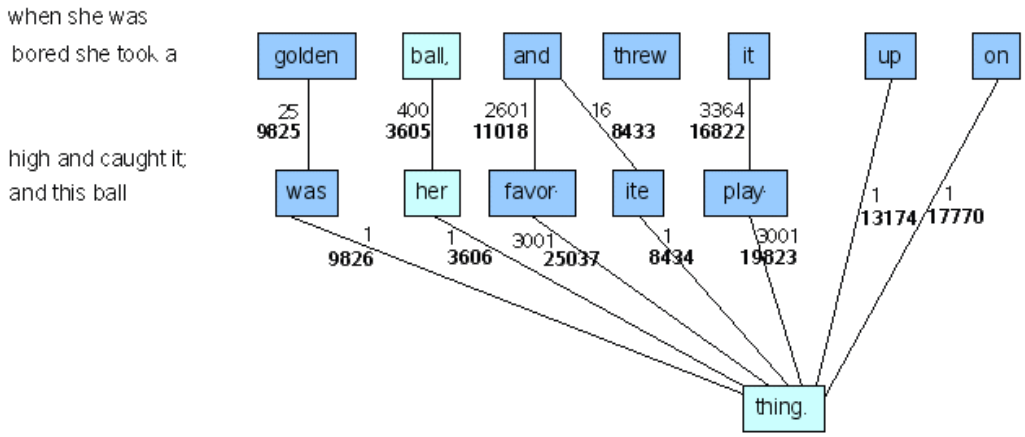


Notice how there is only one pathway leading from each feasible break-point to the preceding break-point; this is the optimal preceding node. If a feasible break-point finally becomes an actual break-point, so will each of the nodes in its path back to the beginning of the paragraph. The end result of the culling leaves a graph that looks like this.



How many lines?

Because this paragraph has been set with comparatively wide lines, and with a tight tolerance, the number of active nodes at any point is quite limited. Loosening the tolerance allows for more "stretch" in the word-spaces; consequently shorter sequences of words become feasible, generally increasing the number of break-points under consideration for a particular line. This translates into an increase in the number of active nodes. Given the tight tolerance of this setting, it is not until the last break-point is considered that the possibility arises of layouts with a varying number of lines. Consider the last part of the graph.



Line 8 must contain the text "when she was bored she took a"; line 9 must contain "high and caught it; and this ball". The last line must contain at least "thing". Feasible break-points for the penultimate line of the paragraph occur in line 8 after both "up" and "on". That is, line 9 contains the rest of the text of the paragraph. The penultimate line-break may also occur in line 9, after "was", "her", "favor-", "favorite" or "play-", the remainder of the paragraph appearing on line 10. The number of lines in the optimal solution is determined entirely by the optimal set of line breaks.

Calculating demerits

Clearly the derivation of the demerits attaching to a feasible break-point is critical to the performance of the algorithm. How is it done? The primary requirement is that the demerits reflect the distance of particular setting of the line from its optimal setting. A line is optimally set when all of its elements, at their natural widths, occupy just the width of the line. This doesn't happen too often, and, except when the margin is "ragged", the text is "justified" to even out the margins. There are two methods by which a line is justified as it is being set. Most adjustment is done by varying the space between words, the *inter-word space*. Although it is also possible to vary the space between letters within a word, the *inter-letter space*, T_EX refrains from this. When this is not enough, the words can be hyphenated. T_EX first attempts to set a paragraph without inserting hyphens, and only resorts to them when no layout is possible within the given parameters.

The main tool for justifying text, then, is the elasticity of inter-word space. Clearly, the degree to which a space can be compressed is less than that to which it can be stretched. The measures noted by K&P for the normal width, maximum shrinkability and maximum stretchability of a word-space are $^1/_3$em, $^1/_9$em and $^1/_6$em. That is, the space in normal text lines can shrink by up to one third, and grow by up to a half. In any given line, the total shrinkage available to the algorithm is the product of the word space allowance and the number of word spaces. Similarly, the total "normal" stretchability is determined by the number of spaces. (This is another reason that it is easier to set text with a simpler vocabulary of shorter words.) While the shrinkage factor is a hard limit, the expansion factor is not, and is affected by the *tolerance* parameter of the algorithm. A tolerance setting of 1 is equivalent to making the stretchability a hard limit; a tolerance of 4 would allow word spaces to stretch to $^4/_6$em, so that the total inter-word space would be up to 1em.

Within these elasticity limits, we can readily measure the divergence of any particular line setting from its optimum, by measuring the total amount of "stretch" or "shrink" in the line, and comparing this to the "normal" elasticity range. Let's say the width available for setting the line is *avl_wid*, and the width of the text with normal word spaces is *txt_wid*. Then the adjustment we need, *adj*, is given by

$$adj = avl_wid - txt_wid$$

If this is zero, we have a perfect fit; if it is positive, we need to stretch the spaces; if it is negative, we need to shrink them. We get our total available shrinkability, *avl_shk*, and total available stretchability, *avl_str*, by multiplying individual shrink and stretch values by the number of word spaces. Now we calculate ajustment ratio, *adj_ratio*.

If $adj <= 0$ $adj_ratio = adj \div avl_shk$.

If $adj > 0$ $adj_ratio = adj \div avl_str$.

If this value is less than -1, the line cannot be set, because the available shrinkability is exhausted. If this occurs when we are testing possible break-points, the break-point is too far from the current

active node, and the node is made inactive. If this value is positive and exceeds the tolerance, the line is too short. If this occurs when testing break-points, the break-point is passed over. If Goldilocks finds a value of 0, the line is just right. The absolute value of the *adj_ratio* (*|adj_ratio|*) is what we use to determine the *badness* of fit of the line setting.

$bad = |adj_ratio|^3 \times 100 + 0{\cdot}5$

NOTE that adjustment ratios, and therefore badness, are always calculated with respect to a *line*; that is, with respect to a pair of break-point nodes.

Penalties

Before continuing with the discussion of demerits, we must become acquainted with the notion of a *penalty*. The archetype of the penalty is the line-break following a hyphen. As long as explicit hyphens remain well-behaved in the middle of other text, they incur no penalty. However, we don't want them appearing immediately before a line-break if we can help it. If they do, they carry a penalty representing this undesirabilty. *Soft hyphens* are those which appear only at the end of a line, when a word must be broken. They are just as undesirable, a fact we indicate by associating the same penalty with them as with manifest hypens. The other thing about soft hyphens is that they take no space in the line unless they appear at the end, unlike manifest hyphens. This diversity is managed by creating a *penalty object* separate from the hyphen (hard or soft) itself, but immediately following it. A penalty has a *value*, representing the desirability or otherwise of a line-break at its location, and a *width*, to accommodate the extra width imposed on a line by a soft hyphen. In the case of a hard hyphen, this is zero.

T_EX makes more extensive use of penalties. The end of a paragraph is marked with a penalty of zero width and a value of *-infinity*, indicating an infinitely desirable break. On the other hand, a penalty of *+infinity* indicates an infinitely undesirable break. T_EX has also introduced a line penalty, usually set to 10, which is included in the calculation of demerits.

Other demerits derive from comparisons between consecutive line endings. If two consecutive lines end with hyphens, a further demerit value is added to those already incurred by the individual hyphens. It is worth noting that this comparison occurs within the optimality horizon of one line, so including this consideration will not disturb the optimality of the solution.

The formulas for demerits

With this information, we can calculate demerits. Given the badness of a line setting, *bad*, calculated as described above, the value of any line-break penalty, *pen*, the consecutive hyphen demerits *hyph*, which will generally be zero, and the line penalty *linep*, then total demerits, *dem*, is calculated using one of three formulae.

$dem = (linep + bad)^2 + pen^2 + hyph$, if $pen >= 0$;

$dem = (linep + bad)^2 - pen^2 + hyph$, if $-infinity < pen < 0$;

$dem = (linep + bad)^2 + hyph$, if last line.

NOTE that demerits, like badness, are always calculated with respect to a *line*; that is, with respect to a pair of break-point nodes.

Over the optimality horizon

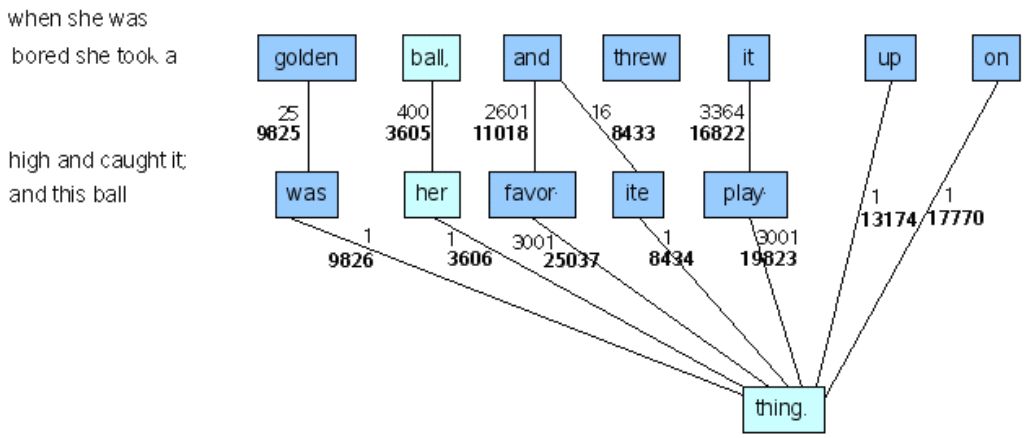
I'm now getting into territory that I *know* I don't understand too well, as opposed to those things I *think* I might have a grasp of. I've always been puzzled by the application of optimality in this algorithm. There are a couple of places where things get a bit murky as far as optimality is concerned; or at least, a little too murky for me to understand readily.

The preceding discussion has been predicated on the unstated assumption that all of the lines are of the same length. Why its that restriction necessary? If we consider the case where all lines are of the same length, which is certainly the most common, it is obvious after a moment's reflection that the actual line on which any particular run of text is laid out can have no influence on the optimal layout for any *preceding* lines. The lines are "anonymous" to the extent that a line of text, once laid out, will look exactly the same on any line. This is better understood in light of a "trick" that K&P recommend for getting a paragraph to be laid out in a given number of lines, when you have reason to believe that such solution will be feasible. Say you want the paragraph to fill *k* lines; you specify line *k*+1 with a length *w* different from all of the other lines, and append an empty box (let's say the

kludge box) of width *w*, followed by a forced line break. As there is no expandable space on the line, this box can only fit on a line exactly *w* wide.

The algorithm as described cannot do this, not simply because of the assumption that all lines are the same width, but because, unless the optimal solution places the last text on line *k* of its own accord, it will be too late by the time it comes to layout the kludge box. In the general case, when lines are of differing lengths, the possibility always exists that a setting which is infeasible on a line of a given length, may find a home on a different line. The algorithm, however, works only with one potential line of text at a time. What we need is some form of feedback from the later occurrence of this event.

There is no feedback, but in the T_EX algorithm there is a form of feed-forward. Consider the problem. Essentially, future conditions may make it necessary for the text to be distributed over a number of lines that is different from the "optimum" solution, as discovered by the one-line sliding window. However, in the standard method, *only the best solution for a particular feasible line-break is remembered*. Consider what happens when we get the the kludge box. There will be a set of active nodes. As these are compared with the break point at the end of the kludge box, only the last, the line break at the end of the actual text, will remain in contention. If the line number of that line doesn't coincide with the one required, too bad. T_EX solves this problem by expanding the set of active nodes carried through the algorithm. Instead of just maintaining the optimum path to each feasible break-point, wherever such a break-point is associated with more than one line number, the optimum path *for each line number* is added to the active nodes. The principle can be illustrated by looking again at the last part of the network diagram.



When the break-point after "thing." is considered, the unadorned method will find that the optimal path passes back through "her" on line 9. All other paths will be forgotten. The modified method, however, will notice that there are active nodes at the end of both line 8 ("up" and "on") and line 9 ("was", "her", favor-", "ite" and "play"). These sets will be optimised separately, resulting in the two possibilities "her" on line 9 and "up" on line 10 with 3606 and 13174 demertis respectively.

Looseness and line counts

The extension to the algorithm has the effect that a given feasible break-point may occur in more than one node. In the basic method, there is a one-to-one correspondence between node and break-point; now the correspondence is between node and break-point/line-number pair. As well as for the "kludge box", this mode is exploited in the *looseness* parameter of K&P's algorithm. *Looseness* defines a variation from the optimal layout discovered by the method. Therefore, if its value is 0 (the default) it has no effect. If not, the algorithm adds the value (which may be negative) to the optimal line-count, and tries to find a solution with the resulting number of lines.

This requires little effort at the end of normal processing; all possible final-break-point/line-count combinations are avalable at the end of processing. What puzzles me, however, is that the problem of laying the paragraph out with a given number of lines, for which the kludge box method was developed, is not simply implemented as a variation of looseness. If the parameter were defined as an signed positive or negative number, or an unsigned number, the signed values would have their current meanings, while the signed number could be read as the desired number of lines. From this distance, I can't see how this would not work, and it would remove the need for the kludge box altogether.

Conditioning nodes with fitness classes

The algorithm attempts to even out differences in the packing density of consecutive lines by tracking this aspect of feasible break-points and applying an extra demerit when the difference exceeds a given threshold. The packing density is divided into four *fitness classes (sic)*. These classes correspond to an adjustment ratio of

```
< -0·5
<= 0·5
<= 1 and
>  1, respectively.
```

Fitness classes are used as follows. Before each possible break-point is considered, variables representing demerits for each of the fitness classes are initialized to *+infinity*. The possible break-point is then compared to each active node in turn, as described elsewhere. As feasible break-points are discovered, both their demerits and their fitness class are calculated.

As we saw above, T_EX tracks break-point/line-number pairs. With fitness classes, it tracks the triplets

break-point/line-number/fitness-class.

However, whereas the line-number distinction is necessary for backtracking from the end of the paragraph when there is an "external" requirement for a number of lines differing from the optimal, the fitness class information is "consumed" in the determination of the next line-break. The determination of the "best" break-point leading to a newly considered feasible break-point is, as always, based on the comparison of total demerits for each path to the new feasible break-point. This calculation now includes the demerits associated with the fitness class of the new line, as compared to the fitness class of the preceding line.

If this analysis is correct, a couple of questions arise. Firstly, why the classes? One immediately noticable effect of the siting of the class boundaries is that all *very loose* lines are grouped together. The influence of *extremely* loose lines is no greater than that of any other line with an adjustment ratio greater than 1.

If instead of fitness classes, demerits for differences in the line packing density of consecutive lines were based directly on the difference in *scaled adjustment ratios*, a threshold could be established beyond which all differences were equivalent. This would have a effect similar to the grouping of all adjustment ratios greater than 1 into the same fitness class. The *scaled adjustment ratios* (`scaled_ratio`) takes account of the difference in scaling for shrink and stretch. Scaling is a means of reducing all adjustment ratios to a linear scale. We define the shrink_scale as the ratio of the normal *inter-word space shrink* factor to the normal *inter-word space stretch* factor.

$$\texttt{shrink_scale} = {}^{1}/_{9} \div {}^{1}/_{6}$$

Then scaling for a line from node *a* to node *b* is achieved by

$$\texttt{scaled_ratio}_{(a\text{->}b)} = \texttt{adj_ratio}_{(a\text{->}b)} < 0 \ ? \ (\ \texttt{adj_ratio}_{(a\text{->}b)} \times \texttt{shrink_scale}\ ) : \texttt{adj_ratio}_{(a\text{->}b)}$$

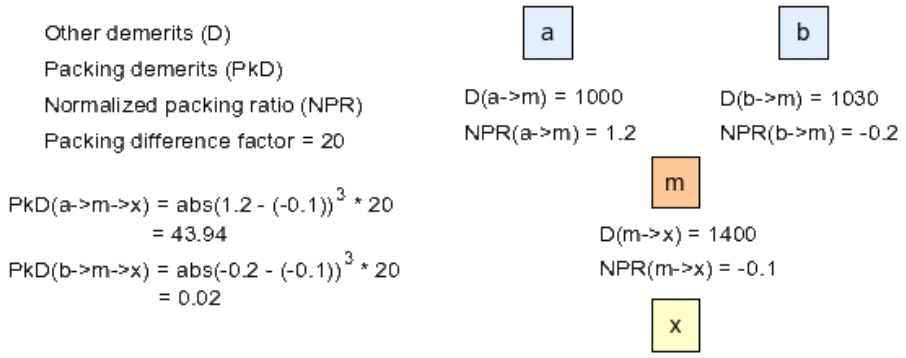
Then, setting the *packing difference threshold*, `pack_diff_thresh`, to 1·5 means that we can calculate the *normalized packing density ratio* for a line from node *a* to node *b* (`n_pack_ratio`) as follows,

$$\texttt{n_pack_ratio}_{(a\text{->}b)} = \texttt{scaled_ratio}_{(a\text{->}b)} > 1{\cdot}5 \ ? \ 1{\cdot}5 : \texttt{scaled_ratio}_{(a\text{->}b)},$$

and this has the same effect as the upper fitness class limit in the T_EX algorithm, in that all differences greater than a certain value are equivalent. If we define a scaling factor *packing difference scaling factor* (`pack_diff_factor`), we can calculate the *packing difference demerits* (`pack_demerits`) between two consecutive lines represented by the node pairs (*a*, *b*) and (*b*, *c*) as

$$\texttt{pack_demerits}(a,\ b,\ c) = (\texttt{abs}(\texttt{n_pack_ratio}_{(a\text{->}b)} - \texttt{n_pack_ratio}_{(b\text{->}c)}))^3 \times \texttt{pack_diff_factor}.$$

`n_pack_ratio` ranges from -0·66 to 1·5; the absolute value of the difference therefore ranges from 0 to 2·1; the cubed value ranges from 0 to 9·26. The `pack_diff_factor` can be chosen to reflect the degree to which packing density variations are frowned upon.



What all this means is that, as we process potential line endings for line *n*, we can directly calculate demerits that take into account the packing density difference between line *n* and any candidate endings for line *n*-1. Consider the above diagram. When the potential lines *(a, m)* and *(b, m)* are considered in isolation, the option *(a, m)* will be chosen on the basis of its lesser demerits. When the line from *(m, x)* is considered, however, the packing difference demerits indicate that the path *(b, m, x)* is preferable to the path *(a, m, x)*. Unfortunately, node *b* is no longer in contention if we abide by a strict one-line optimality horizon.

The upshot, here as in the processing of fitness classes, is that extra active nodes must be installed in line *n*-1, to cater for possibilities which will only become manifest as we process line *n*.

At this point, the primary advantage of the fitness classes becomes apparent. They fix the maximum extra overhead of active nodes to three. To use a continuously varying measure of packing density opens the possibility of many more active nodes. The number of extra nodes can be limited. Consider again the nodes a, b, m and x above. The path *(b, m, x)* is preferred in the end because of the small difference in the *normalized packing ratio* between *(b, m)* and *(m, x)*. For *(b, m, x)* the optimal *normalized packing ratio* for *(m, x)* would be the same as that for *(b, m)*, namely -0.2.

In general, then, to decide whether to retain *b* as an active node, we have only to calculate the *packing difference demerits* for the path *(a, m, K)* where node *K* has the same *normalized packing ratio* as *b*. If the demerit advantage of *a* over *b* is less than this value, retain *b* as an active node.

The Use of Penalties in Folio

The box/glue/penalty model in T_EX

So far I have not discussed the basic model of paragraph contents as used by T_EX. So far, I have assumed line-breaks at "natural" word-break positions, at manifest hyphens, and at "soft" hyphenation positions. The natural word-breaks, in many scripts, occur at inter-word spaces, which are assumed to be able to shrink and stretch from a default base width, within defined limits. When breaks occur at hyphens, they incur a penalty. Extra penalties accrue for consecutive lines ending breaking at hyphens.

T_EX formalizes and extends this model. Unbreakable components of a line are *boxes*, with a given invariant width. Any white space in a line is *glue*, which has a natural width, a specified shrinkage length, and a specified stretch length. *Penalties* have a width (as in the case of a soft hyphen, which only assumes a length if it occurs at a break), and a value. In the case of a hyphen, this is the penalty that is associated with all uses of hyphens. Breaks can occur at glue items following a box, or at a penalty item. Because penalty items carry a cost, they can be used to encourage or discourage a break. To this end, the value they carry can range between *-infinity* and *+infinity*. A value of *-infinity* is an infinitely attractive place to break a line; a value of *+infinity* is an infinitely bad place for a break. So, for example, the end of the paragraph is marked by a penalty with a value of *-infinity*, as a break is compulsory there. Similarly, a break can always be prevented by a penalty of *+infinity*. Because a piece of glue is only eligible as a break when it follows a box, preceding such a piece with a penalty of *+infinity* makes it into a non-breaking glue item. Penalties also carry a convenience kludge known as a *flag*. Consecutive line-breaks at flagged penalty items incur an extra penalty. This is the means by which consecutive lines broken at hyphens receive an extra penalty; hyphen penalties are flagged.

T_EX uses such characteristics to perform a wide variety of formatting, by varying the natural width

and elasticity of glue items, and combining them judiciously with various penalties, the algorithm can be made to massage text into ingenious shapes.

The Folio approach

There are a number of basic differences in the working environment of Folio, which lead to a different approach. Folio is an implementation of the formatting object of the XSL 1.0 and later Recommendations (known as XSL-FO). This is a comprehensive formatting system in which many of the layout problems solved in T_EX with the box/glue/penalty model are specified in detail. While it might be feasible to translate the specifics of XSL-FO into the T_EX model, it seems to me not worth the trouble. In terms of line layout, it is the basic line-breaking model that Folio requires from T_EX.

Folio is also taking a particular approach to rendering. Layout is developed in terms of the Java2D tools, and individual renderers are expected to be able to render the Graphics2D objects of Java2D on the appropriate medium. The basic representation of text in Java2D is the `java.text.AttributedString`. The characters of `AttributedString`s are supported by a map of attributes, and `java.text.AttributedCharacterIterator` provides a number of possibilities for iterating over the text. In this simplified environment, there seems to be no advantage in implementing a full box/glue/penalty model. Break-points in the text can be discovered using an instance of `java.text.BreakIterator`, which serves to delimit boxes and to highlight white-space and both kinds of hyphens. Unicode provides a variety of space characters, some elastic and some fixed, as well as non-breaking space. It also provides a soft-hyphen, which may be explicit in the text, or inserted by the hyphenator. Absent any requirement for "special" penalties, penalty values can simply be added to hard and soft hyphens.

The processing of "flagged penalties" raises another question. These exist to enable consecutive line ending on flagged penalties to accrue an extra penalty. It seems to me useful to generalize this to handle special cases in relationships between consecutive lines. Another instance of "relationship" handling is adjusting for lines whose packing densities vary widely; the cases handled by the fitness classes. When all *in situ* processing of a feasible break has been concluded, the neighbourhood processing can finalize the demerits for the break-point, and perform any related storage management, like adding alternative entries to the active nodes.

Developing the Folio Model

After any hyphenation has been applies to a Folio paragraph, the text of the paragraph can be scanned with a `LineBreak` instance of `BreakIterator` for all of the valid line-break positions, including hyphenation possibilities, in the text. From this raw data, a number of objects are constructed.

BreakPoint

A `BreakPoint` object is constructed for each break point returned from the iterator. The precise method of scanning for break points will be discussed below. The `BreakPoint` is the fundamental object used in the algorithm, and a unique instance is associated with any given break point.

PathSegment

A `PathSegment` corresponds to a segment from a `BreakPoint` which is its end-point, to a preceding `PathSegment` on a path that leads from back to the beginning of the paragraph. Any unique pair of `BreakPoint`s may have more than one `PathSegment`. A `PathSegment` is characterized by three properties:

its end-point
: a `BreakPoint`

its preceding path
: a `PathSegment`

its line number
: the number of `PathSegment`s from its end-point to the beginning of the paragraph.

Although a `PathSegment` may have only one predecessor, it can have multiple successor paths.

Line

The `Line` represents the distance between a pair of `BreakPoint` objects. It is represented as a vector from one `BreakPoint` to a preceding `BreakPoint` in the paragraph. The `Line` object performs the measurements for layout between a pair of `BreakPoints`. It is characterized by three properties:

its end-point
: a `BreakPoint`

its start-point
: a `BreakPoint`

its width
: the available distance between its start-point and end-point.

Note that a given pair of `BreakPoints` may have 0, 1 or more `Lines` associated with them. If 0, there is no viable line between the two. If one, only one distance is defined between the two. If more than one, more than one distance is defined between the two. This can occur when more than one line number is possible between a pair of `BreakPoints`.

Last modified: 23 Jul 2007
Logo by permission of Schoenberg Center for Electronic Text & Image